SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
COMPUTER SCIENCE AND ENGINEERING PROGRAMME

Engineer to Excel

Course Code: ITA04 Course Title: STATISTICS WITH R PROGRAMMING

List of Experiments

1. Write a R program to take input from the user (name and age) and display the values. Also print the version of R installation.

```
name <- readline("Enter your name: ")
age <- as.integer(readline("Enter your age: "))
cat("Name:", name, "\n")
cat("Age:", age, "\n")
print(paste("R version:", R.version$version.string))
```

2. Write a R program to get the details of the objects in memory

```
object_names <- ls()
for (name in object_names) {
object_size <- object.size(get(name))
object_class <- class(get(name))
  cat("Object name:", name, "\n")
  cat("Class:", object_class, "\n")
  cat("Size (bytes):", object_size, "\n\n")
}
```

3. Write a R program to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 91.

```
numbers_seq <- seq(20, 50)
mean_20_to_60 <- mean(20:60)
sum_51_to_91 <- sum(51:91)
cat("Sequence of numbers from 20 to 50:", numbers_seq, "\n")
cat("Mean of numbers from 20 to 60:", mean_20_to_60, "\n")
cat("Sum of numbers from 51 to 91:", sum_51_to_91, "\n")
```

4. Write a R program to create a vector which contains 10 random integer values between -50 and +50.

```
random_vector <- sample(-50:50, 10, replace = TRUE)
cat("Random vector:", random_vector, "\n")
```

5. Write a R program to get the first 10 Fibonacci numbers.

```
n=10
n1=0
n2=1
n3=0
for(i in (1:n)){
cat(n1,"\n")
n3=n1+n2
n1=n2
n2=n3
}
```

6. Write a R program to get all prime numbers up to a given number (based on the sieve of Eratosthenes).

```
n=100
for(i in 2:n){
    count=0
    for(j in 2:(i/2)){
    if(i%%j==0){
            count=count+1
            break}
    }
    if(count==0){
            cat(i,"\n")}
}
```

7. Write a R program to print the numbers from 1 to 100 and print "Fizz" for multiples of 3, print "Buzz" for multiples of 5, and print "FizzBuzz" for multiples of both.

```
for (i in 1:100) {
  if (i %% 3 == 0 && i %% 5 == 0) {
    cat("FizzBuzz", "\n")
  } else if (i %% 3 == 0) {
    cat("Fizz", "\n")
  } else if (i %% 5 == 0) {
    cat("Buzz", "\n")
  } else {
    cat(i, "\n")
  }
}
```

8. Write a R program to extract first 10 english letter in lower case and last 10 letters in upper case and extract letters between 22nd to 24th letters in upper case.

```
first_10_lower <- letters[1:10]
last_10_upper <- toupper(letters[17:26])
between_22_24_upper <- toupper(letters[22:24])
cat("First 10 English letters in lowercase:", first_10_lower, "\n")
cat("Last 10 letters in uppercase:", last_10_upper, "\n")
cat("Letters between 22nd and 24th positions in uppercase:", between_22_24_upper, "\n")
```

9. Write a R program to find the factors of a given number.

```
n=50
for(i in 1:n){
if(n%%i==0){
 cat(i,"\n")}}
```

10. Write a R program to find the maximum and the minimum value of a given vector.

```
vector <- c(5, 2, 10, 8, 3, 6)
max_value <- max(vector)
min_value <- min(vector)
print(max_value)
print(min_value)
```

11. Write a R program to get the unique elements of a given string and unique numbers of vector.

```
string <- "Hello, World!"
vector <- c(5, 2, 10, 8, 3, 6, 2, 5, 8, 10)
unique_elements <- unique(strsplit(string, "")[[1]])
unique_numbers <- unique(vector)
cat("Unique elements of the string:", unique_elements, "\n")
cat("Unique numbers of the vector:", unique_numbers, "\n")
```

12. Write a R program to create three vectors a,b,c with 3 integers. Combine the three vectors to become a 3×3 matrix where each column represents a vector. Print the content of the matrix.

```
a <- c(1, 2, 3)
b <- c(4, 5, 6)
c <- c(7, 8, 9)
matrix_combined <- cbind(a, b, c)
print(matrix_combined)
```

List of Experiments

13. Write a R program to create a list of random numbers in normal distribution and count occurrences of each value.

```
random_list <- rnorm(100, mean = 0, sd = 1)
occurrences <- table(random_list)
cat("Occurrences of each value:\n")
print(occurrences)
```

14. Write a R program to read the .csv file and display the content.

```
data <- read.csv("filename.csv")
print(data)
```

15. Write a R program to create three vectors numeric data, character data and logical data. Display the content of the vectors and their type.

```
numeric_vector <- c(1, 2, 3, 4, 5)
character_vector <- c("apple", "banana", "orange", "grape", "kiwi")
logical_vector <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
print(numeric_vector)
print(character_vector)
cat("Type of numeric vector:", class(numeric_vector), "\n")
cat("Type of character vector:", class(character_vector), "\n")
cat("Type of logical vector:", class(logical_vector), "\n")
```

16. Write a R program to create a 5 x 4 matrix , 3 x 3 matrix with labels and fill the matrix by rows and 2 × 2 matrix with labels and fill the matrix by columns.

```
matrix_rows <- matrix(1:20, nrow = 5, ncol = 4, byrow = TRUE)
row.names(matrix_rows) <- c("Row 1", "Row 2", "Row 3", "Row 4", "Row 5")
print(matrix_rows)
matrix_cols <- matrix(1:9, nrow = 3, ncol = 3, byrow = FALSE)
colnames(matrix_cols) <- c("Column 1", "Column 2", "Column 3")
print(matrix_cols)
matrix_cols_2x2 <- matrix(1:4, nrow = 2, ncol = 2, byrow = FALSE)
colnames(matrix_cols_2x2) <- c("Column A", "Column B")
print(matrix_cols_2x2)
```

17. Write a R program to create an array, passing in a vector of values and a vector of dimensions. Also provide names for each dimension.

```
values <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
dimensions <- c(3, 2, 2)
array_data <- array(values, dim = dimensions, dimnames = list(c("A", "B", "C"), c("X", "Y"), c("P", "Q")))
cat("Array:\n")
print(array_data)
```

18. Write a R program to create an array with three columns, three rows, and two "tables", taking two vectors as input to the array. Print the array.

```
vector1 <- c(1, 2, 3, 4, 5, 6, 7, 8, 9)
vector2 <- c(10, 11, 12, 13, 14, 15, 16, 17, 18)
array_data <- array(c(vector1, vector2), dim = c(3, 3, 2))
cat("Array:\n")
print(array_data)
```

19. Write a R program to create a list of elements using vectors, matrices and a functions. Print the content of the list.

```
vector1 <- c(1, 2, 3)
vector2 <- c("apple", "banana", "orange")
vector3 <- c(TRUE, FALSE, TRUE)
matrix1 <- matrix(1:4, nrow = 2)
matrix2 <- matrix(5:8, nrow = 2)
my_function <- function(x) {
  return(x^2)
}
my_list <- list(vector1, vector2, vector3, matrix1, matrix2, my_function)
print(my_list)
```

20. Write a R program to draw an empty plot and an empty plot specify the axes limits of the graphic

```
plot.new()
# Draw an empty plot with specified axes limits
plot.new()
plot.window(xlim = c(0, 10), ylim = c(0, 100))
```

21. Write a R program to create an array of two 3x3 matrices each with 3 rows and 3 columns from two given two vectors. Print the second row of the second matrix of the array and the element in the 3rd row and 3rd column of the 1st matrix.

```
vector1 <- c(1, 2, 3, 4, 5, 6, 7, 8, 9)

vector2 <- c(10, 11, 12, 13, 14, 15, 16, 17, 18)

array_data <- array(c(matrix(vector1, nrow = 3), matrix(vector2, nrow = 3)), dim = c(3, 3, 2))

cat("Second row of the second matrix:\n")

print(array_data[2, , 2])

cat("Element in the third row and third column of the first matrix:\n")

print(array_data[3, 3, 1])
```

22. Write a R program to combine three arrays so that the first row of the first array is followed by the first row of the second array and then first row of the third array.

```
array1 <- array(1:9, dim = c(3, 3))

array2 <- array(10:18, dim = c(3, 3))

array3 <- array(19:27, dim = c(3, 3))

combined_array <- rbind(array1[1, , ], array2[1, , ], array3[1, , ])

print(combined_array)
```

23. Write a R program to create an array using four given columns, three given rows, and two given tables and display the content of the array.

```
columns <- 4
rows <- 3
tables <- 2
array_data <- array(1:(columns * rows * tables), dim = c(rows, columns, tables))
print(array_data)
```

List of Experiments

24. Write a R program to create a two-dimensional 5x3 array of sequence of even integers greater than 50.

```
array_data <- array(seq(52, by = 2, length.out = 5*3), dim = c(5, 3))
print(array_data)
```

25. Create below data frame

```
exam _data = data.frame(
name = c('Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael', 'Matthew', 'Laura', 'Kevin',
'Jonas').
score = c(12.5, 9, 16.5. 12, 9, 20. 14.5, 13.5, 8, 19),
attempts = c(1, 3. 2, 3, 2, 3. 1, 1, 2, 1),
qualify = c('yes'. 'no'. 'yes'. 'no'. 'no'. 'yes'. 'yes'. 'no'. 'no'. 'yes')
)
```

Course Code: ITA04 Course Title: STATISTICS WITH R PROGRAMMING

a. Write a R program to extract 3rd and 5th rows with 1st and 3rd columns from a given data frame

```
df <- data.frame(
  col1 = c(1, 2, 3, 4, 5),
  col2 = c(6, 7, 8, 9, 10),
  col3 = c(11, 12, 13, 14, 15)
)
extracted_data <- df[c(3, 5), c(1, 3)]
print(extracted_data)
```

b. Write a R program to add a new column named country in a given data frame

```
Country<-c("USA","USA","USA","USA","UK","USA","USA","India","USA","USA")
df <- data.frame(
  name = c("John", "Alice", "Mark"),
  score = c(80, 90, 75)
)
df$country <- c("USA", "Canada", "UK")
print(df)
```

c. Write a R program to add new row(s) to an existing data frame

```
new_exam_data = data.frame(name = c('Robert', 'Sophia'),score = c(10.5, 9), attempts = c(1, 3),qualify = c('yes', 'no'))
df <- data.frame(
  name = c("John", "Alice"),
  score = c(80, 90)
)
```

```
new_exam_data <- data.frame(
  name = c("Robert", "Sophia"),
  score = c(10.5, 9),
  attempts = c(1, 3),
  qualify = c("yes", "no")
)
updated_df <- rbind(df, new_exam_data)
print(updated_df)
```

d. Write a R program to sort a given data frame by name and score

```
df <- data.frame(
  name = c("John", "Alice", "Mark"),
  score = c(80, 90, 75)
)
sorted_df <- df[order(df$name, df$score), ]
print(sorted_df)
```

e. Write a R program to save the information of a data frame in a file and display the information of the file.

```
df <- data.frame(

  name = c("John", "Alice", "Mark"),
```

```
 score = c(80, 90, 75)


)


file_path <- "data.csv"


write.csv(df, file = file_path, row.names = FALSE)


file_content <- readLines(file_path)


cat("File Content:\n")


cat(file_content, sep = "\n")
```

26. Write a R program to call the (built-in) dataset airquality. Check whether it is a data frame or not? Order the entire data frame by the first and second column. remove the variables 'Solar.R' and 'Wind' and display the data frame.

```
data(airquality)

if (is.data.frame(airquality)) {

 cat("The 'airquality' dataset is a data frame.\n")

} else {

 cat("The 'airquality' dataset is not a data frame.\n")

}
```

```
ordered_df <- airquality[order(airquality$Month, airquality$Day), ]

# Remove 'Solar.R' and 'Wind' variables

cleaned_df <- subset(ordered_df, select = -c(Solar.R, Wind))

print(cleaned_df)
```

27. Write a R program to create a factor corresponding to height of women data set , which inbuild in R, contains height and weights for a sample of women.

```
data(women)
height_factor <- factor(women$height)
print(height_factor)
```

List of Experiments

28. Write a R program to extract the five of the levels of factor created from a random sample from the LETTERS (Part of the base R distribution.)

```
sample_letters <- sample(LETTERS, size = 20, replace = TRUE)

factor_letters <- factor(sample_letters)

# Extract five levels from the factor

five_levels <- levels(factor_letters)[1:5]

print(five_levels)
```

29. **Iris** dataset is a very famous dataset in almost all data mining, machine learning courses, and it has been an R build-in dataset. The dataset consists of 50 samples from each of three species of Iris flowers (Iris setosa, Iris virginica and Iris versicolor). Four features(variables) were measured from each sample, they are the **length** and the **width** of sepal and petal, in centimetres. Perform the following EDA steps .

Course Code: ITA04 Course Title: STATISTICS WITH R PROGRAMMING

(i)Find dimension, Structure, Summary statistics, Standard Deviation of all features.
(ii)Find mean and standard deviation of features groped by three species of Iris flowers (Iris setosa, Iris virginica and Iris versicolor)
(iii)Find quantile value of sepal width and length
(iV)create new data frame named iris1 which have a new column name **Sepal.Length.Cate** that categorizes "Sepal.Length" by quantile
(V) Average value of numerical varialbes by two categorical variables: Species and Sepal.Length.Cate:
(vi) Average mean value of numerical varialbes by Species and Sepal.Length.Cate
(vii)Create Pivot Table based on Species and Sepal.Length.Cate.

```
# Load the Iris dataset
data(iris)
# (i) Find dimension, structure, summary statistics, standard deviation of all features
cat("Dimension of the dataset:", dim(iris), "\n\n")
cat("Structure of the dataset:\n")
str(iris)
cat("\nSummary statistics of all features:\n")
summary(iris)
cat("\nStandard deviation of all features:\n")
sapply(iris, sd)
cat("\n")

# (ii) Find mean and standard deviation of features grouped by three species of Iris flowers
cat("Mean and standard deviation of features grouped by species:\n")
aggregate(iris[, 1:4], by = list(iris$Species), FUN = function(x) c(Mean = mean(x), SD = sd(x)))
cat("\n")

# (iii) Find quantile value of sepal width and length
cat("Quantile value of sepal width and length:\n")
quantile(iris$Sepal.Width)
quantile(iris$Sepal.Length)
cat("\n")
```

```
# (iv) Create new data frame named iris1 with a new column Sepal.Length.Cate that categorizes
"Sepal.Length" by quantile
iris1 <- iris
iris1$Sepal.Length.Cate <- cut(iris1$Sepal.Length, breaks = quantile(iris1$Sepal.Length), labels =
FALSE, include.lowest = TRUE)
cat("New data frame 'iris1' with Sepal.Length.Cate:\n")
print(iris1)
cat("\n")

# (v) Average value of numerical variables by two categorical variables: Species and
Sepal.Length.Cate
cat("Average value of numerical variables by Species and Sepal.Length.Cate:\n")
aggregate(iris1[, 1:4], by = list(iris1$Species, iris1$Sepal.Length.Cate), FUN = mean)
cat("\n")

# (vi) Average mean value of numerical variables by Species and Sepal.Length.Cate
cat("Average mean value of numerical variables by Species and Sepal.Length.Cate:\n")
aggregate(iris1[, 1:4], by = list(iris1$Species, iris1$Sepal.Length.Cate), FUN = function(x)
mean(mean(x)))
cat("\n")

# (vii) Create Pivot Table based on Species and Sepal.Length.Cate
cat("Pivot Table based on Species and Sepal.Length.Cate:\n")
pivot_table <- aggregate(iris1[, 1:4], by = list(iris1$Species, iris1$Sepal.Length.Cate), FUN = mean)
pivot_table <- reshape2::dcast(pivot_table, Group.1 ~ Group.2, value.var = c("Sepal.Length",
"Sepal.Width", "Petal.Length", "Petal.Width"))
print(pivot_table)
```

30. Randomly Sample the iris dataset such as 80% data for training and 20% for test and create Logistics regression with train data, use species as target and petals width and length as feature variables , Predict the probability of the model using test data, Create Confusion matrix for above test model

```
data(iris)
train_indices <- sample(1:nrow(iris), 0.8 * nrow(iris))
train_data <- iris[train_indices, ]
test_data <- iris[-train_indices, ]
model <- glm(Species ~ Petal.Width + Petal.Length, data = train_data, family = binomial)
probabilities <- predict(model, newdata = test_data, type = "response")
predicted_species <- ifelse(probabilities > 0.5, "versicolor", "setosa")
confusion_matrix <- table(predicted_species, test_data$Species)
print(confusion_matrix)
```

31. (i)Write suitable R code to compute the mean, median ,mode of the following values c(90, 50, 70, 80, 70, 60, 20, 30, 80, 90, 20)

(ii) Write R code to find 2nd highest and 3rd Lowest value of above problem.

```
values <- c(90, 50, 70, 80, 70, 60, 20, 30, 80, 90, 20)
mean_value <- mean(values)
median_value <- median(values)
mode_value <- names(table(values))[table(values) == max(table(values))]
cat("Mean:", mean_value, "\n")
cat("Median:", median_value, "\n")
cat("Mode:", mode_value, "\n\n")
# (ii) Find the 2nd highest and 3rd lowest values
sorted_values <- sort(unique(values))
second_highest <- sorted_values[length(sorted_values) - 1]
third_lowest <- sorted_values[3]
cat("2nd highest value:", second_highest, "\n")
cat("3rd lowest value:", third_lowest, "\n")
```

32. Explore the airquality dataset. It contains daily air quality measurements from New York during a period of five months:

• Ozone: mean ozone concentration (ppb), • Solar.R: solar radiation (Langley),
• Wind: average wind speed (mph), • Temp: maximum daily temperature in degrees Fahrenheit,
• Month: numeric month (May=5, June=6, and so on),• Day: numeric day of the month (1-31).

i. Compute the mean temperature(don't use build in function) ii.Extract the first five rows from airquality.

iii.Extract all columns from airquality except Temp and Wind iv.Which was the coldest day during the period?

v.How many days was the wind speed greater than 17 mph?

```
# Load the airquality dataset
data(airquality)

# i. Compute the mean temperature
mean_temperature <- sum(airquality$Temp) / nrow(airquality)
cat("Mean Temperature:", mean_temperature, "\n\n")

# ii. Extract the first five rows from airquality
first_five_rows <- airquality[1:5, ]
print(first_five_rows)
cat("\n")

# iii. Extract all columns from airquality except Temp and Wind
selected_columns <- airquality[, !(names(airquality) %in% c("Temp", "Wind"))]
print(selected_columns)
cat("\n")

# iv. Find the coldest day during the period
coldest_day <- airquality[which.min(airquality$Temp), ]
cat("Coldest Day:\n")
print(coldest_day)
cat("\n")

# v. Count the number of days with wind speed greater than 17 mph
wind_gt_17_count <- sum(airquality$Wind > 17, na.rm = TRUE)
cat("Number of days with wind speed greater than 17 mph:", wind_gt_17_count, "\n")
```

33. (i)Get the Summary Statistics of air quality dataset

(ii)Melt airquality data set and display as a long – format data?

(iii)Melt airquality data and specify month and day to be "ID variables"?

(iv)Cast the molten airquality data set with respect to month and date features

(v) Use cast function appropriately and compute the average of Ozone, Solar.R , Wind and temperature per month?

```
# Load the airquality dataset
data(airquality)

# (i) Get the Summary Statistics of airquality dataset
summary_stats <- summary(airquality)
print(summary_stats)
cat("\n")

# (ii) Melt airquality dataset and display as a long-format data
library(reshape2)
melted_data <- melt(airquality)
print(melted_data)
cat("\n")

# (iii) Melt airquality dataset and specify month and day as "ID variables"
melted_data_with_id <- melt(airquality, id.vars = c("Month", "Day"))
print(melted_data_with_id)
cat("\n")

# (iv) Cast the molten airquality dataset with respect to month and date features
casted_data <- dcast(melted_data_with_id, Month + Day ~ variable)
print(casted_data)
cat("\n")

# (v) Compute the average of Ozone, Solar.R, Wind, and Temperature per month
averages_per_month <- cast(melted_data_with_id, Month ~ variable, mean)
print(averages_per_month)
```

34.(i) Find any missing values(na) in features and drop the missing values if its less than 10% else replace that with mean of that feature.

(ii) Apply a linear regression algorithm using Least Squares Method on “Ozone” and “Solar.R” (iii)Plot Scatter plot between Ozone and Solar and add regression line created by above model

```
# Load the airquality dataset
data(airquality)

# (i) Find missing values and replace or drop them
missing_count <- colSums(is.na(airquality))
missing_percent <- missing_count / nrow(airquality) * 100

# Check if missing values are less than 10%
if (any(missing_percent < 10)) {
  # Replace missing values with mean of the respective feature
  airquality[is.na(airquality)] <- lapply(airquality, function(x) {
    ifelse(is.na(x), mean(x, na.rm = TRUE), x)
  })
} else {
  # Drop rows with missing values
  airquality <- na.omit(airquality)
}

# (ii) Apply linear regression using Least Squares Method on "Ozone" and "Solar.R"
model <- lm(Ozone ~ Solar.R, data = airquality)

# (iii) Plot Scatter plot between Ozone and Solar.R with regression line
plot(airquality$Solar.R, airquality$Ozone, xlab = "Solar.R", ylab = "Ozone", main = "Scatter Plot")
abline(model, col = "red")
```

35. Load dataset named ChickWeight,

(i).Order the data frame, in ascending order by feature name “weight” grouped by feature “diet” and Extract the last 6 records from order data frame.

(ii).a Perform melting function based on “Chick", "Time", "Diet" features as ID variables

b. Perform cast function to display the mean value of weight grouped by Diet

c. Perform cast function to display the mode of weight grouped by Diet

```
# Load the ChickWeight dataset
data(ChickWeight)

# (i) Order the data frame by "weight" in ascending order, grouped by "diet" and extract the last 6 records
ordered_data <- ChickWeight[order(ChickWeight$weight), ]
last_six_records <- tail(ordered_data, 6)

# (ii.a) Perform melting function on "Chick", "Time", and "Diet" as ID variables
melted_data <- reshape2::melt(ChickWeight, id.vars = c("Chick", "Time", "Diet"))

# (ii.b) Perform cast function to display the mean value of "weight" grouped by "Diet"
mean_weight <- reshape2::dcast(melted_data, Diet ~ ., mean, value.var = "weight")

# (ii.c) Perform cast function to display the mode of "weight" grouped by "Diet"
mode_weight <- reshape2::dcast(melted_data, Diet ~ ., function(x) {
  names(table(x))[which.max(table(x))]
}, value.var = "weight")
```

36. a. Create Box plot for “weight” grouped by “Diet”

b. Create a Histogram for “weight” features belong to Diet- 1 category

c. Create Scatter plot for “ weight” vs “Time” grouped by Diet

```
# Load the ChickWeight dataset
data(ChickWeight)

# (a) Create Box plot for "weight" grouped by "Diet"
boxplot(ChickWeight$weight ~ ChickWeight$Diet, main = "Box plot of Weight by Diet", xlab = "Diet", ylab = "Weight")

# (b) Create a Histogram for "weight" for Diet 1 category
subset_data <- ChickWeight[ChickWeight$Diet == 1, ]
```

```
hist(subset_data$weight, main = "Histogram of Weight for Diet 1", xlab = "Weight", ylab = "Frequency")

# (c) Create Scatter plot for "weight" vs "Time" grouped by Diet
plot(ChickWeight$Time, ChickWeight$weight, col = ChickWeight$Diet, pch = 16, main = "Scatter plot of
Weight vs Time", xlab = "Time", ylab = "Weight")
legend("topright", legend = unique(ChickWeight$Diet), col = unique(ChickWeight$Diet), pch = 16, title =
"Diet")
```

37. a. Create multi regression model to find a weight of the chicken , by “Time” and “Diet” as as predictor variables
    b. Predict weight for Time=10 and Diet=1
    c. Find the error in model for same

```
# Fit the multiple regression model
model <- lm(weight ~ Time + Diet, data = ChickWeight)

# (a) Print the model summary
summary(model)

# (b) Predict weight for Time=10 and Diet=1
new_data <- data.frame(Time = 10, Diet = 1)
predicted_weight <- predict(model, newdata = new_data)
cat("Predicted weight for Time=10 and Diet=1:", predicted_weight, "\n")

# (c) Find the error in the model for the same prediction
actual_weight <- ChickWeight$weight[ChickWeight$Time == 10 & ChickWeight$Diet == 1]
error <- actual_weight - predicted_weight
cat("Error in the model:", error, "\n")
```

List of Experiments

38. .For this exercise, use the (built-in) dataset Titanic.
    a. Draw a Bar chart to show details of “Survived” on the Titanic based on passenger Class
    b. Modify the above plot based on gender of people who survived
    c. Draw histogram plot to show distribution of feature “Age”

```
# Load the Titanic dataset
data(Titanic)

# (a) Bar chart of "Survived" based on passenger class
barplot(Titanic$Survived, beside = TRUE, col = c("red", "green"),
      legend = c("Did not survive", "Survived"),
```

```
    names.arg = c("1st Class", "2nd Class", "3rd Class"),
    main = "Survival on Titanic by Passenger Class",
    xlab = "Passenger Class", ylab = "Count")

# (b) Bar chart of "Survived" based on passenger class and gender
barplot(Titanic, beside = TRUE, col = c("red", "green"),
    legend = c("Did not survive", "Survived"),
    args.legend = list(x = "topleft"),
    main = "Survival on Titanic by Passenger Class and Gender",
    xlab = "Passenger Class and Gender", ylab = "Count")

# (c) Histogram of "Age" distribution
hist(Titanic$Age, breaks = 20,
   main = "Distribution of Age on Titanic",
   xlab = "Age", ylab = "Count")
```

39. Explore the USArrests dataset, contains the number of arrests for murder, assault, and rape for each of the 50 states in 1973. It also contains the percentage of people in the state who live in an urban area.
    (i) a. Explore the summary of Data set, like number of Features and its type. Find the number of records for each feature. Print the statistical feature of data
    b. Print the state which saw the largest total number of rape
    c. Print the states with the max & min crime rates for murder
    (ii).a. Find the correlation among the features
    b. Print the states which have assault arrests more than median of the country
    c. Print the states are in the bottom 25% of murder
    (iii). a. Create a histogram and density plot of murder arrests by US stat
    b. Create the plot that shows the relationship between murder arrest rate and proportion of the population that is urbanised by state. Then enrich the chart by adding assault arrest rates (by colouring the points from blue (low) to red (high)). c. Draw a bar graph to show the murder rate for each of the 50 states .

```
# Load the USArrests dataset
data(USArrests)

# (i) (a) Summary of the dataset
summary(USArrests)

# (b) State with the largest total number of rape
max_rape_state <- rownames(USArrests)[which.max(USArrests$Rape)]
```

```
cat("State with the largest total number of rape:", max_rape_state, "\n")

# (c) States with max and min crime rates for murder
max_murder_states <- rownames(USArrests)[which.max(USArrests$Murder)]
min_murder_states <- rownames(USArrests)[which.min(USArrests$Murder)]
cat("States with the max crime rate for murder:", max_murder_states, "\n")
cat("States with the min crime rate for murder:", min_murder_states, "\n")

# (ii) (a) Correlation among the features
correlation <- cor(USArrests)
print(correlation)

# (b) States with assault arrests more than the median of the country
median_assault <- median(USArrests$Assault)
states_above_median <- rownames(USArrests)[USArrests$Assault > median_assault]
cat("States with assault arrests more than the median:", states_above_median, "\n")

# (c) States in the bottom 25% of murder
bottom_25_states <- rownames(USArrests)[USArrests$Murder <= quantile(USArrests$Murder, 0.25)]
cat("States in the bottom 25% of murder:", bottom_25_states, "\n")

# (iii) (a) Histogram and density plot of murder arrests
hist(USArrests$Murder, main = "Histogram of Murder Arrests", xlab = "Murder Arrests", ylab =
"Frequency")
lines(density(USArrests$Murder), col = "red")

# (b) Scatter plot of murder arrest rate vs. urban population proportion, colored by assault arrest rate
plot(USArrests$UrbanPop, USArrests$Murder, xlab = "Urban Population Proportion", ylab = "Murder
Arrest Rate",
    main = "Murder Arrest Rate vs. Urban Population Proportion", col = USArrests$Assault, pch = 19)
legend("bottomright", legend = c("Low Assault Rate", "High Assault Rate"), col = c("blue", "red"), pch =
19)

# (c) Bar graph of murder rate for each state
barplot(USArrests$Murder, names.arg = rownames(USArrests), xlab = "State", ylab = "Murder Rate",
      main = "Murder Rate by State", col = "darkblue")
```

40. 4. a. Create a data frame based on below table.

Month	1	2	3	4	5	6	7	8	9	10	11	12
Spends	1000	4000	5000	4500	3000	4000	9000	11000	15000	12000	7000	3000
Sales	9914	40487	54324	50044	34719	42551	94871	118914	158484	131348	78504	36284

b. Create a regression model for that data frame table to show the amount of sales(Sales) based on the how much the company spends (Spends) in advertising c. Predict the Sales if Spend=13500

```
# Create the data frame
df <- data.frame(
  Month = 1:12,
  Spends = c(1000, 4000, 5000, 4500, 3000, 4000, 9000, 11000, 15000, 12000, 7000, 3000),
  Sales = c(9914, 40487, 54324, 50044, 34719, 42551, 94871, 118914, 158484, 131348, 78504, 36284)
)

# (b) Create a regression model for Sales based on Spends
model <- lm(Sales ~ Spends, data = df)

# (c) Predict the Sales if Spend=13500
new_data <- data.frame(Spends = 13500)
predicted_sales <- predict(model, newdata = new_data)

# Print the predicted sales
cat("Predicted Sales for Spend = 13500:", predicted_sales, "\n")
```